A

Special Project

Project Requirement Specification (PRS)

on

AI Voice Controlled ERP Assistant

by

Pranamya Mani Tripathi (24STUCHH010777)

Gone Srishanth Reddy (24STUCHH010428)

Harshulchandhra (24STUCHH010172)

Chintakindi Badrinath Goud (24STUCHH010308)

Under the guidance of

Sowjanya Ramisetty Mam

Faculty of Science & Technology,

The ICFAI Foundation for Higher Education, Hyderabad

Submitted on or before 26/07/2026

Project Requirement Specification (PRS)

Special Project: AI Voice-Controlled ERP Assistant

Item	Details
Document	Project Requirement Specification (Phase 1 of the Special Project)
Team	Pranamya Mani Tripathi, Gone Srishanth Reddy, Harshulchandhra, Chintakindi Badrinath Goud
Supervisor	Sowjanya Ramisetty Mam
Institution	Faculty of Science & Technology, The ICFAI Foundation for Higher Education, Hyderabad
Submission deadline	26/07/2026

1. Project Title

AI Voice-Controlled ERP Assistant: A Natural Language Interface for ERP Operations.

The project designs and builds an assistant through which users operate Enterprise Resource Planning (ERP) functions (inventory, employees, attendance and sales) by speaking or typing plain-language commands instead of navigating screens, menus and forms.

2. Problem Statement

ERP systems hold the operational data of an organisation, yet they are difficult to use. Even a routine task, such as checking the stock of one item, requires navigating several screens, filling forms and knowing the menu hierarchy. This makes everyday work slow and error-prone, consumes disproportionate time on repetitive operations, and creates a steep learning curve for non-technical users.

Existing speech recognition and natural language understanding research is mostly evaluated in isolation on benchmarks. It seldom delivers a complete and safe path from a spoken command to an actual ERP database operation. General conversational frameworks are flexible but heavy, and they offer no ready-made safeguards such as mandatory confirmation before a data write. Users must also normally know exact record codes, while in everyday speech they refer to records by name (for example, “hydraulic valve”).

Core question: how can users perform common ERP operations through voice or text in plain language, on an ordinary laptop, while keeping data writes safe and intentional?

3. Aim and Objectives

3.1 Aim

To design and implement an AI Voice-Controlled ERP Assistant that lets users perform ERP functions using natural language, by voice or text, instead of manual navigation, and that changes data only with the user's explicit confirmation.

3.2 Objectives

- Design a layered architecture (user interface, AI processing, integration, ERP core) with a clear responsibility for each layer.
- Implement intent recognition and entity extraction using a rule-based engine by default, with an optional LLM-based engine (Ollama with Llama 3.2) for natural phrasing variation and automatic fallback to rules.
- Build a dialogue manager that asks for missing information, confirms uncertain entity matches and requires explicit confirmation before every write operation.
- Provide fuzzy entity resolution so that users can refer to items and employees by name rather than exact code.
- Support voice input (Whisper or Vosk) and speech output (pyttsx3) through pluggable engines, alongside text input.
- Evaluate the prototype for correctness (intent accuracy and entity resolution accuracy), usability and response time.

3.3 Functional Requirements

ID	Requirement
FR1	Accept a command as text or as recorded voice through the web interface.
FR2	Convert speech to text through a pluggable STT engine (Whisper or Vosk); text input bypasses STT.
FR3	Identify the intent (stock query, attendance mark/retrieve, sales report, employee information) and extract its entities.
FR4	Ask a follow-up question when a required entity is missing (for example, "Check stock" then "Which item?").
FR5	Resolve names to ERP records by fuzzy matching and ask for confirmation when a match is uncertain.
FR6	Require an explicit "yes" before any write (for example, marking attendance); no write without confirmation.
FR7	Route validated requests through the ERP gateway (FastAPI REST endpoints for text and audio) to the SQLite ERP.
FR8	Return results as text, with optional speech output through pyttsx3.
FR9	Provide a role-based access hook in the gateway and optional login with hashed credentials.

3.4 Non-Functional Requirements

- **Performance:** acceptable response time on a standard CPU-only laptop.
- **Safety:** zero database writes without user confirmation.
- **Modularity:** STT, TTS and NLU engines swappable without changing the core system.
- **Portability:** runs on a Windows laptop with a microphone and speakers; frontend needs no build step.
- **Extensibility:** the mock database can later be replaced by a production ERP such as Odoo or a PostgreSQL-backed system.

4. Scope of the Project

4.1 In Scope

- Web interface (HTML, CSS, JavaScript) with a text box and a voice record button, and a FastAPI backend with REST endpoints for text and audio.
- Mock SQLite ERP with four modules: Inventory, Employees, Attendance and Sales.
- Use cases: stock query; attendance marking and retrieval (with confirmation); sales report by date range (for example, “Get total sales from last month”); employee information; and the multi-turn clarification flow.
- Rule-based NLU (default) and optional Ollama + Llama 3.2 NLU with fallback; fuzzy entity resolution; role-based access hook; optional login using hashed credentials.
- Deliverables: working source code, demo script with interactive mode, web interface, project report and presentation slides.

4.2 Out of Scope (Future Work)

- Connection to a production ERP (Odoo, SAP or PostgreSQL), full OAuth2/JWT authentication, rate limiting and security hardening.
- Multilingual support, proactive alerts (low stock, attendance anomalies) and Docker/cloud deployment.

4.3 System Overview

Every request passes through a seven-stage pipeline: **input** → **speech-to-text** → **NLU** → **dialogue manager** → **ERP gateway** → **database query** → **response** (text, with optional speech output). Text input enters the pipeline directly at the NLU stage.

4.4 Assumptions and Constraints

- The prototype uses synthetic data only; no live organisational data is involved.
- A CPU-only laptop is the target; the 1B Llama 3.2 model and text mode keep resource needs low.
- Internet is needed only for optional Whisper API use and one-time model downloads.
- English is the only supported language in this phase.

5. Data Requirements

No external dataset is required. The project uses a synthetic SQLite database that simulates an ERP, together with a set of test commands and speech samples for evaluation.

5.1 ERP Database (Synthetic SQLite)

Module	Main data fields (proposed)	Purpose
Inventory	item code (e.g. P-101, P-102), item name, category, quantity in stock, unit price	Stock queries; name-to-code fuzzy matching
Employees	employee ID, name, department, designation, contact details, role, hashed password	Employee information; login and role checks
Attendance	attendance ID, employee ID, date, check-in time, status	Mark attendance (write, with confirmation) and retrieve records
Sales	sale ID, item code, quantity, amount, sale date	Sales totals and reports by date range

The seed data should contain enough sample items (including similar-sounding names to test fuzzy matching), employees, attendance records and sales spread across several months so that date-range reports such as “last month” return meaningful results.

5.2 Input Data

- **Text commands:** a labelled set of test commands for each intent, including phrasing variations, missing-entity cases (“Check stock”), and confirmation replies (“yes”/“no”). Example: “Show stock for item P-102”, “Mark my attendance”.
- **Voice data:** short recorded audio commands (WAV or browser-recorded format) for testing Whisper/Vosk, including item codes and item names, which are known weak points of speech recognition.
- **Entity data:** item names, item codes and employee names used by the fuzzy matcher, with deliberately misspelt or misheard variants (for example, “hydraulic valve” mapped to its item code).

5.3 Output and Evaluation Data

- **Responses:** text replies (and optional synthesised speech) returned by the assistant.
- **Evaluation records:** expected versus actual intent and entities per test command, entity resolution results, and response time measured for both the rule-based and LLM NLU modes.
- **Logs:** request/response logs of the dialogue flows for debugging and for verifying that no write occurred without confirmation.

5.4 Data Handling and Privacy

- All data is synthetic, so no personal or confidential information is used.
- Passwords (optional login) are stored only as hashed credentials.
- Write operations are executed only after explicit user confirmation.

5.5 Data Requirement Summary

Data type	Source	Volume (proposed)
ERP records	Synthetic SQLite database created by the team	Small seed set per module, sufficient for all use cases
Test commands	Written by the team for each intent and flow	Multiple phrasings per intent
Audio samples	Recorded by the team through microphone	Short clips covering the core commands
Models	Whisper or Vosk (STT); Llama 3.2 1B via Ollama (optional NLU)	One-time download